G2++ Model Due Diligence

> ***Providing code scaffolding for swaption price calibration***
>
> *The user seems to be asking for code related to calibration and swaption pricing, likely without needing online data. We might have G2++ and existing MATLAB functions available for calibration, implied volatility conversion, and price wrapping. I'll provide function signatures, including a possible wrapper around the current swaption price.*
>
> ***Setting up swaption calibration with market targets***
>
> *I'll need to keep this code sharp and focused — using market targets for calibration, with conversion functions and the Black/Bachelier formulas. For swaption pricing, I'll make sure to include both formulas for normal and lognormal volatility. I'll also ensure it can handle forward swap rates, with multi-curve support where necessary for precision.*
>
> *Thought for 18s*

The immediate goal should be a calibration harness, not a perfect architecture. We can treat the current swaption pricer as a black-box function, then wrap market-vol conversion, objective construction, and parameter updates around it.

Yes. Before Prototype 2, you can build a **calibration harness around the current class**.

The rule:

$$\boxed{\text{Do not refactor first. Wrap what exists.}}$$

We need four helper blocks:

$$\text{market vol} \leftrightarrow \text{market price}$$

$$\theta_{\text{raw}} \mapsto \theta_{\text{model}}$$

$$\text{model swaption price wrapper}$$

$$\text{calibration objective}$$

---

1. Calibration object

For calibration, use a flat struct array, not a matrix directly.

A matrix is good for display:

$$\text{expiry} \times \text{tenor}$$

but the optimizer wants a vector of instruments.

Use:

```
calibSet(k).expiry
calibSet(k).paymentDates
calibSet(k).strike
calibSet(k).marketVol
calibSet(k).volType       % 'normal' or 'black'
calibSet(k).direction     % 'PAY' or 'REC'
calibSet(k).notional
calibSet(k).weight
```

Then a surface matrix becomes:

```
k = 0;
for i = 1:numel(expiries)
    for j = 1:numel(tenors)
        k = k + 1;
        calibSet(k).expiry = expiries(i);
        calibSet(k).paymentDates = paymentDateCells{i,j};
        calibSet(k).strike = strikes(i,j);
        calibSet(k).marketVol = marketVols(i,j);
        calibSet(k).volType = 'normal';
        calibSet(k).direction = 'PAY';
        calibSet(k).notional = 1.0;
        calibSet(k).weight = 1.0;
    end
end
```

Keep date-generation separate for now. Do not mix schedule construction with calibration.

2. Volatility to price

You need market prices because the G2++ closed-form gives a price.

Then either calibrate on prices:

$$P_k^{model}(\theta) - P_k^{market}$$

or on implied vols:

$$\sigma_k^{model}(\theta) - \sigma_k^{market}.$$

For a first calibration, use **price errors normalized by annuity**:

$$\epsilon_k(\theta) = \frac{P_k^{model}(\theta) - P_k^{market}}{N_k A_k}.$$

This avoids solving implied vol at every optimizer step.

swaptionPriceFromVol.m

```
function price = swaptionPriceFromVol(F, K, T, annuity, vol, volType, directio
n, notional)
% Converts quoted swaption volatility to price.
%
% F         forward swap rate
% K         strike
% T         option expiry in years
% annuity   swap annuity at t=0
% vol       quoted normal or Black vol
% volType   'normal' or 'black'
% direction 'PAY' or 'REC'
% notional  scalar

    arguments
        F (1,1) double
        K (1,1) double
        T (1,1) double {mustBeNonnegative}
        annuity (1,1) double {mustBeNonnegative}
        vol (1,1) double {mustBeNonnegative}
        volType (1,:) char {mustBeMember(volType, {'normal','black'})}
        direction (1,:) char {mustBeMember(direction, {'PAY','REC'})} = 'PAY'
        notional (1,1) double = 1.0
    end

    omega = 1;
    if strcmpi(direction, 'REC')
        omega = -1;
    end

    if T == 0 || vol == 0
        intrinsic = max(omega * (F - K), 0);
        price = notional * annuity * intrinsic;
        return;
    end

    sqrtT = sqrt(T);

    switch lower(volType)
        case 'normal'
            std = vol * sqrtT;
            d = (F - K) / std;

            price = notional * annuity * ...
```

```
                (omega * (F - K) * normcdf(omega * d) + std * normpdf(d));

        case 'black'
            if F <= 0 || K <= 0
                error('Black volatility requires strictly positive F and K.');
            end

            std = vol * sqrtT;
            d1 = (log(F / K) + 0.5 * std^2) / std;
            d2 = d1 - std;

            price = notional * annuity * ...
                omega * (F * normcdf(omega * d1) - K * normcdf(omega * d2));
    end
end
```

In current EUR rates, normal/Bachelier is often safer because it tolerates negative rates.

---

3. Price to implied volatility

Use this only for diagnostics at first.

swaptionImpliedVolFromPrice.m

```
function impVol = swaptionImpliedVolFromPrice(price, F, K, T, annuity, volType,
direction, notional)
% Computes implied normal or Black volatility from a swaption price.

    arguments
        price (1,1) double {mustBeNonnegative}
        F (1,1) double
        K (1,1) double
        T (1,1) double {mustBeNonnegative}
        annuity (1,1) double {mustBeNonnegative}
        volType (1,:) char {mustBeMember(volType, {'normal','black'})}
        direction (1,:) char {mustBeMember(direction, {'PAY','REC'})} = 'PAY'
        notional (1,1) double = 1.0
    end

    omega = 1;
    if strcmpi(direction, 'REC')
        omega = -1;
    end

    intrinsic = notional * annuity * max(omega * (F - K), 0);

    if price < intrinsic - 1e-12
        impVol = NaN;
        return;
    end

    if abs(price - intrinsic) < 1e-14
        impVol = 0;
        return;
    end

    objFun = @(v) swaptionPriceFromVol(F, K, T, annuity, v, volType, direction,
notional) - price;

    switch lower(volType)
        case 'normal'
            volLow = 1e-10;
            volHigh = 5.0;      % 500% absolute normal vol, intentionally wide

        case 'black'
            if F <= 0 || K <= 0
                impVol = NaN;
                return;
            end
```

```
            volLow = 1e-10;
            volHigh = 5.0;
    end

    while objFun(volHigh) < 0
        volHigh = 2 * volHigh;
        if volHigh > 100
            impVol = NaN;
            return;
        end
    end

    impVol = fzero(objFun, [volLow, volHigh]);
end
```

Use this for reporting:

```
modelImpVol = swaptionImpliedVolFromPrice(modelPrice, F, K, T, annuity, volTyp
e, direction, notional);
```

Do not make this the first calibration objective unless the price objective is already stable.

---

4. Forward swap rate and annuity

For a single-curve approximation:

$$A_0 = \sum_{i=1}^{n} \delta_i P(0, T_i),$$

$$F_0 = \frac{P(0, T_0) - P(0, T_n)}{A_0}.$$

Helper:

`swaptionForwardAndAnnuity.m`

```
function [F, annuity, delta, P0Ti, P0T0] = swaptionForwardAndAnnuity(model, exp
iry, paymentDates)
% Computes forward swap rate and annuity using the model's initial curve.
%
% expiry and paymentDates are assumed to be year fractions.

    arguments
        model (1,1) GaussianTwoFactors
        expiry (1,1) double
        paymentDates (1,:) double
    end

    paymentDates = paymentDates(:);

    allDates = [expiry; paymentDates];
    zc = model.getInitialMarketZC(allDates.');
    zc = zc(:);

    P0T0 = zc(1);
    P0Ti = zc(2:end);

    delta = diff([expiry; paymentDates]);

    annuity = sum(delta .* P0Ti);

    F = (P0T0 - P0Ti(end)) / annuity;
end
```

This is enough for first calibration.

Later, for multi-curve setup, forward projection should use the forecast curve and annuity discounting should use the discount curve.

---

5. Parameter transformation

For calibration, never let the optimizer touch raw parameters directly.

Use:

$$a = e^{\alpha}, \qquad b = e^{\beta}, \qquad \sigma_i = e^{u_i}, \qquad \eta_i = e^{v_i}, \qquad \rho = \tanh z.$$

`g2ppApplyRawParams.m`

```
function params = g2ppApplyRawParams(model, thetaRaw, nBuckets)
% Maps unconstrained optimizer variables to admissible G2++ parameters.

    arguments
        model (1,1) GaussianTwoFactors
        thetaRaw (:,1) double
        nBuckets (1,1) double {mustBeInteger, mustBePositive}
    end

    expectedLength = 3 + 2 * nBuckets;

    if numel(thetaRaw) ~= expectedLength
        error('Expected thetaRaw length %d, got %d.', expectedLength, numel(the
taRaw));
    end

    params.a = exp(thetaRaw(1));
    params.b = exp(thetaRaw(2));
    params.rho = tanh(thetaRaw(3));

    params.sigma = exp(thetaRaw(4 : 3 + nBuckets));
    params.eta   = exp(thetaRaw(4 + nBuckets : 3 + 2*nBuckets));

    model.meanReversion_x = params.a;
    model.meanReversion_y = params.b;
    model.correlation = params.rho;
    model.volatility_sigma = params.sigma(:).';
    model.volatility_eta = params.eta(:).';
end
```

This mutates the model object. That is acceptable for prototype calibration.

Production version should be cleaner, but this gets you working.

`g2ppInverseRawParams.m`

```
function thetaRaw = g2ppInverseRawParams(a, b, rho, sigma, eta)
% Maps admissible G2++ parameters to unconstrained optimizer variables.

    arguments
        a (1,1) double {mustBePositive}
        b (1,1) double {mustBePositive}
        rho (1,1) double
        sigma (:,1) double {mustBePositive}
        eta (:,1) double {mustBePositive}
    end

    if abs(rho) >= 1
        error('rho must satisfy abs(rho) < 1.');
    end

    thetaRaw = [
        log(a);
        log(b);
        atanh(rho);
        log(sigma(:));
        log(eta(:))
    ];
end
```

6. Model price wrapper

This assumes you have or will soon have:

```
model.priceSwaption(strike, expiry, paymentDates, ...)
```

If your current method name is different, only change this wrapper.

`g2ppSwaptionModelPrice.m`

```
function price = g2ppSwaptionModelPrice(model, instr)
% Thin wrapper around the current G2++ swaption pricer.

    arguments
        model (1,1) GaussianTwoFactors
        instr (1,1) struct
    end

    if isfield(instr, 'direction')
        direction = instr.direction;
    else
        direction = 'PAY';
    end

    if isfield(instr, 'notional')
        notional = instr.notional;
    else
        notional = 1.0;
    end

    % Adjust this line to the exact current method signature.
    price = model.priceSwaption( ...
        instr.strike, ...
        instr.expiry, ...
        instr.paymentDates, ...
        'direction', direction, ...
        'notional', notional);
end
```

If you do **not** yet have final `priceSwaption`, temporarily replace the last block with a placeholder that errors clearly:

```
error('priceSwaption is not implemented yet. Calibration wrapper is ready but c
annot run.');
```

7. Calibration residual vector

This is the central helper.

g2ppCalibrationResiduals.m

```
function [residuals, diagnostics] = g2ppCalibrationResiduals(thetaRaw, model, c
alibSet, nBuckets, args)
% Returns residual vector for G2++ swaption calibration.
%
% Supports calibration on:
%   - 'price'
%   - 'normalizedPrice'
%   - 'vol'

    arguments
        thetaRaw (:,1) double
        model (1,1) GaussianTwoFactors
        calibSet (1,:) struct
        nBuckets (1,1) double {mustBeInteger, mustBePositive}
        args.objectiveType (1,:) char {mustBeMember(args.objectiveType, {'pric
e','normalizedPrice','vol'})} = 'normalizedPrice'
        args.badValuePenalty (1,1) double = 1e6
    end

    g2ppApplyRawParams(model, thetaRaw, nBuckets);

    n = numel(calibSet);
    residuals = zeros(n,1);

    diagnostics.modelPrice = NaN(n,1);
    diagnostics.marketPrice = NaN(n,1);
    diagnostics.modelVol = NaN(n,1);
    diagnostics.marketVol = NaN(n,1);
    diagnostics.forward = NaN(n,1);
    diagnostics.annuity = NaN(n,1);

    for k = 1:n
        instr = calibSet(k);

        try
            [F, annuity] = swaptionForwardAndAnnuity(model, instr.expiry, inst
r.paymentDates);

            marketPrice = swaptionPriceFromVol( ...
                F, instr.strike, instr.expiry, annuity, ...
                instr.marketVol, instr.volType, instr.direction, instr.notiona
l);

            modelPrice = g2ppSwaptionModelPrice(model, instr);
```

```
            diagnostics.modelPrice(k) = modelPrice;
            diagnostics.marketPrice(k) = marketPrice;
            diagnostics.marketVol(k) = instr.marketVol;
            diagnostics.forward(k) = F;
            diagnostics.annuity(k) = annuity;

            switch args.objectiveType
                case 'price'
                    residuals(k) = sqrt(instr.weight) * (modelPrice - marketPri
ce);

                case 'normalizedPrice'
                    scale = max(instr.notional * annuity, 1e-12);
                    residuals(k) = sqrt(instr.weight) * (modelPrice - marketPri
ce) / scale;

                case 'vol'
                    modelVol = swaptionImpliedVolFromPrice( ...
                        modelPrice, F, instr.strike, instr.expiry, annuity, ...
                        instr.volType, instr.direction, instr.notional);

                    diagnostics.modelVol(k) = modelVol;

                    if ~isfinite(modelVol)
                        residuals(k) = args.badValuePenalty;
                    else
                        residuals(k) = sqrt(instr.weight) * (modelVol - instr.m
arketVol);
                    end
            end

            if ~isfinite(residuals(k))
                residuals(k) = args.badValuePenalty;
            end

        catch
            residuals(k) = args.badValuePenalty;
        end
    end
end
```

For the first run, use:

```
    'objectiveType', 'normalizedPrice'
```

not vol.

8. Calibration driver

`g2ppCalibrateToSwaptionSet.m`

```
function result = g2ppCalibrateToSwaptionSet(model, calibSet, theta0Raw, nBucke
ts, args)
% Calibrates current G2++ model to a flat swaption calibration set.

    arguments
        model (1,1) GaussianTwoFactors
        calibSet (1,:) struct
        theta0Raw (:,1) double
        nBuckets (1,1) double {mustBeInteger, mustBePositive}
        args.objectiveType (1,:) char {mustBeMember(args.objectiveType, {'pric
e','normalizedPrice','vol'})} = 'normalizedPrice'
        args.maxIterations (1,1) double = 200
        args.display (1,:) char = 'iter'
        args.useLsqnonlin (1,1) logical = true
    end

    residualFun = @(theta) g2ppCalibrationResiduals( ...
        theta, model, calibSet, nBuckets, ...
        'objectiveType', args.objectiveType);

    if args.useLsqnonlin
        opts = optimoptions('lsqnonlin', ...
            'Display', args.display, ...
            'MaxIterations', args.maxIterations, ...
            'StepTolerance', 1e-8, ...
            'FunctionTolerance', 1e-8);

        [thetaStar, resnorm, residuals, exitflag, output] = lsqnonlin(residualF
un, theta0Raw, [], [], opts);

    else
        objFun = @(theta) sum(residualFun(theta).^2);

        opts = optimset( ...
            'Display', args.display, ...
            'MaxIter', args.maxIterations);

        [thetaStar, resnorm, exitflag, output] = fminsearch(objFun, theta0Raw,
opts);
        residuals = residualFun(thetaStar);
```

```
    end

    g2ppApplyRawParams(model, thetaStar, nBuckets);

    [finalResiduals, diagnostics] = g2ppCalibrationResiduals( ...
        thetaStar, model, calibSet, nBuckets, ...
        'objectiveType', args.objectiveType);

    result.thetaRaw = thetaStar;
    result.resnorm = resnorm;
    result.residuals = residuals;
    result.finalResiduals = finalResiduals;
    result.exitflag = exitflag;
    result.output = output;
    result.diagnostics = diagnostics;

    result.params.a = model.meanReversion_x;
    result.params.b = model.meanReversion_y;
    result.params.rho = model.correlation;
    result.params.sigma = model.volatility_sigma;
    result.params.eta = model.volatility_eta;

    result.rmse = sqrt(mean(finalResiduals.^2));
    result.maxAbsError = max(abs(finalResiduals));
end
```

9. Minimal usage

```
nBuckets = numel(model.volatility_sigma);

theta0Raw = g2ppInverseRawParams( ...
    model.meanReversion_x, ...
    model.meanReversion_y, ...
    model.correlation, ...
    model.volatility_sigma(:), ...
    model.volatility_eta(:));

result = g2ppCalibrateToSwaptionSet( ...
    model, ...
    calibSet, ...
    theta0Raw, ...
    nBuckets, ...
    'objectiveType', 'normalizedPrice', ...
    'useLsqnonlin', true);
```

Then inspect:

```
result.params
result.rmse
result.maxAbsError
result.diagnostics
```

10. Price objective versus volatility objective

For first calibration:

$$\boxed{\text{use normalized price residuals.}}$$

Reason:

$$\sigma^{model}$$

requires numerical inversion at every optimizer call. That is slow and can fail for bad intermediate parameters.

Better first objective:

$$\epsilon_k = \frac{P_k^{model} - P_k^{market}}{N_k A_k}.$$

Then after calibration, compute implied vols for reporting:

$$\sigma_k^{model} = \mathrm{ImpVol}(P_k^{model}).$$

Only later calibrate directly on vol errors.

---

11. Three robustness tests

Test 1 — volatility/price round trip

Purpose:

$$\sigma \to P \to \sigma$$

should recover the input volatility.

```
F = 0.025;
K = 0.027;
T = 5;
A = 4.2;
vol = 0.006; % normal vol
notional = 1.0;

price = swaptionPriceFromVol(F, K, T, A, vol, 'normal', 'PAY', notional);

impVol = swaptionImpliedVolFromPrice(price, F, K, T, A, 'normal', 'PAY', notion
al);

assert(abs(impVol - vol) < 1e-10);
```

This tests the market conversion layer independently from G2++.

Test 2 — self-calibration sanity check

Generate synthetic market prices/vols from the model itself.

Procedure:

1. Choose parameters $\theta^\star$.
2. Price all calibration swaptions with the model.
3. Convert model prices to market implied vols.
4. Start calibration close to $\theta^\star$.
5. Check objective goes near zero.

Expected:

$$\mathrm{RMSE} \approx 0.$$

This test proves your calibration harness is wired correctly.

It does **not** prove the model formula is financially correct, but it proves the optimizer and wrappers are coherent.

Test 3 — bad parameter robustness

Feed deliberately bad raw parameters.

Example:

```
thetaBad = theta0Raw;
thetaBad(3) = 20;       % rho approx 1
thetaBad(4:end) = -30;  % near-zero vol
```

Then:

```
res = g2ppCalibrationResiduals(thetaBad, model, calibSet, nBuckets);
assert(all(isfinite(res)));
```

The objective must not crash during optimization.

It should return finite penalties.

---

12. Minimal current implementation order

Do this in order:

$$\boxed{1.\ \texttt{swaptionPriceFromVol}}$$

$$\boxed{2.\ \texttt{swaptionImpliedVolFromPrice}}$$

$$\boxed{3.\ \texttt{swaptionForwardAndAnnuity}}$$

$$\boxed{4.\ \texttt{g2ppApplyRawParams / g2ppInverseRawParams}}$$

$$\boxed{5.\ \texttt{g2ppSwaptionModelPrice}}$$

$$\boxed{6.\ \texttt{g2ppCalibrationResiduals}}$$

$$\boxed{7.\ \texttt{g2ppCalibrateToSwaptionSet}}$$

Then run the three tests.

The calibration scaffold can be ready even if the final `priceSwaption` method is still being corrected. The only hard dependency is that `g2ppSwaptionModelPrice` must call whatever current method produces one G2++ swaption price.

Response:

5/2/2026, 6:32:46 PM

I'll keep this as an implementation audit, not a full refactor. The decisive point: `priceSwaptionG2PP` can be added now only if we patch `getSwaptionInstrument`, `getSwaptionBondBasket`, `muTForward`, `getLambdaAndBy`, and `yBarSolver`.

Good. Immediate objective:

$$\boxed{\text{make one reliable } \texttt{priceSwaptionG2PP} \text{ method before calibration.}}$$

The uploaded class already contains bond reconstruction, bucketed variance, forward-state moments, and partial swaption machinery, but several current functions are fatal or semi-fatal for swaption pricing. `% Gaussian Two-Fator Model aka G2++.txt`

---

1. Fatal bugs / blockers in the current file

A. Class boundary is broken

You currently have:

```
end
end

function output = muTForward(...)
```

This means several functions after the class ending are **local file functions**, not class methods.

Fatal consequence:

```
obj.varTForward(...)
obj.covTForward(...)
obj.getSwaptionInstrument(...)
```

will not work if those functions are outside the `methods ... end` block.

Fix: move every intended method inside:

```
methods
    ...
end
```

before the final `end` of the class.

B. `muTForward` appears twice

There is one version inside the class and another outside. This is dangerous.

Fix: keep one version only.

Also, the formula currently returns:

```
output = sum(muComp1 + muComp2 + muComp3);
```

but the comment says:

$$\mu_x^T(s,t) = -M_x^T(s,t).$$

For the actual mean under the T-forward measure, use:

```
output = sum(-muComp1 + muComp2 + muComp3);
```

Same logic for the y-process by symmetry.

C. `args.tMaturity` is required, not optional

Current:

```
args.tMaturity (1,1) double;
```

Then:

```
if ~isempty(args.tMaturity)
```

This is inconsistent. MATLAB will require `tMaturity`.

Fix:

```
args.tMaturity (1,1) double = NaN;
```

Then:

```
if isnan(args.tMaturity)
    tMaturity = endDate;
else
    tMaturity = args.tMaturity;
end
```

D. `varI` is called but not defined

Current method name:

```
vari(obj, startDate, endDate)
```

But later:

```
obj.varI(0, currMaturity)
```

Fatal.

Fix: add alias:

```
function output = varI(obj, startDate, endDate)
    output = obj.vari(startDate, endDate);
end
```

Keep this now. Rename later.

E. `getSwaptionBondBasket` rejects yearfrac inputs

Current logic errors unless both inputs are datenums:

```
if (currMaturity > 1000) && (ulSwapSteps(1) > 1000)
    ...
else
    error("currMaturity and ulSwapSteps in different format")
end
```

Fatal for calibration, because you will usually pass yearfracs.

Fix: allow both datenums and yearfracs.

F. `getSwaptionInstrument` is incomplete

It does not return:

```
output.coeffs
output.delta
output.expiry
output.paymentDates
output.direction
output.notional
```

But `getLambdaAndBy` expects:

```
instr.coeffs
```

Fatal.

G. `getLambdaAndBy` computes `x` but ignores it

Current:

```
x = x_state(:);
...
lambda = coeffs .* A0Ti .* exp(-Bx0Ti .* x_state);
```

Should be:

```
lambda = coeffs .* A0Ti .* exp(-Bx0Ti .* x);
```

H. `yBarSolver` uses undefined `scalarF`

Current:

```
yfs = fzero(scalarF, y0, args.option);
```

Fatal.

Also `args.option = optimoptions('fsolve',...)` is not correct for `fzero`.

Use a scalar helper and bracketed `fzero`.

I. `getInitialMarketZC` uses forecast curve

Current:

```
mktZC = obj.DCF.forecast.getDf(t, 'basisTenor', obj.basis);
```

For a first G2++ short-rate discounting model, the safer convention is:

```
obj.DCF.discount.getDf(...)
```

But do not change this blindly if the current library intentionally uses forecast curve in HW1F. For now flag as **model convention risk**, not fatal.

---

2. Minimal methods needed before

`priceSwaptionG2PP`

You need these methods, in this order:

```
varI
getSwaptionInstrument
getSwaptionBondBasket
getLambdaAndBy
getYBarValue
yBarSolver
getStateGaussianAtExpiry
getConditionalYMoments
priceSwaptionG2PP
```

The final swaption formula will use:

$$P(0,T_0)\mathbb{E}^{T_0}\left[\left(1-\sum_i c_i P(T_0,T_i)\right)^+\right]$$

for payer, with:

$$P(T_0,T_i)=A_i\exp(-B_i^x x-B_i^y y).$$

---

3. Drop-in method 1 - `varI`

Put this inside the class `methods` block.

```
function output = varI(obj, startDate, endDate)
    % Alias for the integrated variance of the short-rate stochastic part:
    %
    %     Var( integral_startDate^endDate [x(u)+y(u)] du )
    %
    % Kept for compatibility with swaption-related formulas.

    arguments
        obj (1,1) GaussianTwoFactors;
        startDate (1,1) double;
        endDate   (1,1) double;
    end

    output = obj.vari(startDate, endDate);
end
```

---

4. Drop-in method 2 - fixed `getSwaptionInstrument`

This gives the bond-basket coefficients.

For a payer swaption:

$$\left[1 - \sum_i c_i P(T_0, T_i)\right]^+,$$

with:

$$c_i = K\delta_i, \quad i < n,$$

$$c_n = 1 + K\delta_n.$$

```
function output = getSwaptionInstrument(obj, strike, currMaturity, ulSwapSteps,
args)
    % Builder of the Swaption Contract Specifications.
    %
    % INPUTS:
    % strike       | double | scalar : swaption strike
    % currMaturity | double | scalar : swaption expiry T0, datenum or yearfrac
    % ulSwapSteps  | double | vector : payment dates [T1,...,Tn], datenum or ye
arfrac
    %
    % OPTIONAL INPUTS:
    % direction | char   | scalar : 'PAY' or 'REC'
    % notional  | double | scalar : default 1e6
    %
    % OUTPUT:
    % output.expiry       | double | scalar
    % output.paymentDates | double | column vector
    % output.delta        | double | column vector
    % output.coeffs       | double | column vector
    % output.strike       | double | scalar
    % output.direction    | char
    % output.notional     | double

    arguments
        obj (1,1) GaussianTwoFactors;
        strike (1,1) double;
        currMaturity (1,1) double;
        ulSwapSteps (1,:) double;
        args.direction (1,:) char {mustBeMember(args.direction, {'PAY','REC'})}
= 'PAY';
        args.notional (1,1) double = 1e6;
    end

    pricingDate_temp = datenum(obj.pricingDate, 'dd/mm/yyyy');

    % Date formatting
    if (currMaturity > 1000) && (ulSwapSteps(1) > 1000)
        currMaturity = yearfracExtend(pricingDate_temp, currMaturity, obj.basi
s);
        ulSwapSteps = yearfracExtend(pricingDate_temp, ulSwapSteps, obj.basis);
    elseif (currMaturity <= 1000) && (ulSwapSteps(1) <= 1000)
        % already year fractions
    else
        error("currMaturity and ulSwapSteps must use the same date format");
    end

    ulSwapSteps = ulSwapSteps(:);
```

```
        if any(ulSwapSteps <= currMaturity)
            error("All underlying swap payment dates must be after currMaturity.");
        end

        delta = diff([currMaturity; ulSwapSteps]);

        coeffs = strike .* delta;
        coeffs(end) = coeffs(end) + 1.0;

        output.expiry = currMaturity;
        output.paymentDates = ulSwapSteps;
        output.delta = delta;
        output.coeffs = coeffs;
        output.strike = strike;
        output.direction = upper(args.direction);
        output.notional = args.notional;
    end
```

5. Drop-in method 3 - fixed `getSwaptionBondBasket`

```
function output = getSwaptionBondBasket(obj, currMaturity, ulSwapSteps)
    % Builds the affine G2++ bond basket:
    %
    %     P(T0,Ti) = A(T0,Ti) * exp(-Bx(T0,Ti)*x - By(T0,Ti)*y)

    arguments
        obj (1,1) GaussianTwoFactors;
        currMaturity (1,1) double;
        ulSwapSteps (1,:) double;
    end

    if isnumeric(obj.pricingDate)
        pricingDate_temp = obj.pricingDate;
    elseif isstring(obj.pricingDate) || ischar(obj.pricingDate)
        pricingDate_temp = datenum(char(obj.pricingDate), 'dd/mm/yyyy');
    else
        error("pricingDate must be numeric datenum, datetime, char, or strin
g");
    end

    if (currMaturity > 1000) && (ulSwapSteps(1) > 1000)
        currMaturity = yearfracExtend(pricingDate_temp, currMaturity, obj.basi
s);
        ulSwapSteps = yearfracExtend(pricingDate_temp, ulSwapSteps, obj.basis);
    elseif (currMaturity <= 1000) && (ulSwapSteps(1) <= 1000)
        % already year fractions
    else
        error("currMaturity and ulSwapSteps must use the same date format");
    end

    ulSwapSteps = ulSwapSteps(:);
    nPayDates = numel(ulSwapSteps);

    zc = obj.getInitialMarketZC([currMaturity, ulSwapSteps.']);
    zc = zc(:);

    P0T0 = zc(1);
    P0Ti = zc(2:end);

    dt = ulSwapSteps - currMaturity;

    Bx0Ti = obj.getH(obj.meanReversion_x, dt);
    By0Ti = obj.getH(obj.meanReversion_y, dt);
```

```
    var0T0 = obj.varI(0, currMaturity);

    varT0Ti = zeros(nPayDates, 1);
    var0Ti = zeros(nPayDates, 1);

    for i = 1:nPayDates
        Ti = ulSwapSteps(i);
        varT0Ti(i) = obj.varI(currMaturity, Ti);
        var0Ti(i) = obj.varI(0, Ti);
    end

    A0Ti = (P0Ti ./ P0T0) .* exp(0.5 .* (varT0Ti - var0Ti + var0T0));

    output.P0T0 = P0T0;
    output.P0Ti = P0Ti;
    output.A0Ti = A0Ti(:);
    output.Bx0Ti = Bx0Ti(:);
    output.By0Ti = By0Ti(:);
    output.expiry = currMaturity;
    output.paymentDates = ulSwapSteps;
end
```

---

6. Drop-in method 4 - fixed `getLambdaAndBy`

```
function [lambda, By] = getLambdaAndBy(obj, strike, currMaturity, ulSwapSteps,
x_state, args)
    % Returns:
    %
    %     lambda_i(x) = c_i * A(T0,Ti) * exp(-Bx(T0,Ti)*x)
    %
    % and By(T0,Ti).

    arguments
        obj (1,1) GaussianTwoFactors;
        strike (1,1) double;
        currMaturity (1,1) double;
        ulSwapSteps (1,:) double;
        x_state (1,1) double = 0;
        args.direction (1,:) char {mustBeMember(args.direction, {'PAY','REC'})}
= 'PAY';
        args.notional (1,1) double = 1e6;
    end

    instr = obj.getSwaptionInstrument( ...
        strike, currMaturity, ulSwapSteps, ...
        'direction', args.direction, ...
        'notional', args.notional);

    basket = obj.getSwaptionBondBasket(currMaturity, ulSwapSteps);

    coeffs = instr.coeffs(:);
    A0Ti = basket.A0Ti(:);
    Bx0Ti = basket.Bx0Ti(:);
    By0Ti = basket.By0Ti(:);

    x = x_state;

    lambda = coeffs .* A0Ti .* exp(-Bx0Ti .* x);
    By = By0Ti;
end
```

---

7. Drop-in method 5 - scalar Y-bar value

Needed because `fzero` needs a scalar-valued function.

```
function value = getYBarValue(obj, y, lambda, By)
    % Scalar value of:
    %
    %     f(y) = 1 - sum_i lambda_i exp(-By_i y)

    arguments
        obj (1,1) GaussianTwoFactors;
        y (1,1) double;
        lambda (:,1) double;
        By (:,1) double;
    end

    value = 1 - sum(lambda .* exp(-By .* y));
end
```

Keep your existing derivative function if `newtonVector` works.

---

8. Drop-in method 6 - robust `yBarSolver`

This version uses bracketed `fzero`. Since $f(y)$ is monotone increasing, bracketing is robust.

```
function yBar = yBarSolver(obj, strike, currMaturity, ulSwapSteps, x_state, arg
s)
    % Solves for yBar(x):
    %
    %     1 = sum_i c_i A(T0,Ti) exp(-Bx_i*x - By_i*yBar)

    arguments
        obj (1,1) GaussianTwoFactors;
        strike (1,1) double;
        currMaturity (1,1) double;
        ulSwapSteps (1,:) double;
        x_state (1,1) double = 0;
        args.direction (1,:) char {mustBeMember(args.direction, {'PAY','REC'})}
= 'PAY';
        args.notional (1,1) double = 1e6;
        args.ySeed (1,1) double = 0;
        args.maxExpansion (1,1) double = 25;
        args.tolerance (1,1) double = 1e-10;
    end

    [lambda, By] = obj.getLambdaAndBy( ...
        strike, currMaturity, ulSwapSteps, x_state, ...
        'direction', args.direction, ...
        'notional', args.notional);

    lambda = lambda(:);
    By = By(:);

    scalarF = @(y) obj.getYBarValue(y, lambda, By);

    yLow = args.ySeed - 0.01;
    yHigh = args.ySeed + 0.01;

    fLow = scalarF(yLow);
    fHigh = scalarF(yHigh);

    expansion = 0;

    while fLow * fHigh > 0 && expansion < args.maxExpansion
        width = yHigh - yLow;
        yLow = yLow - 2 * width;
        yHigh = yHigh + 2 * width;

        fLow = scalarF(yLow);
        fHigh = scalarF(yHigh);

        expansion = expansion + 1;
    end
```

```
    if fLow * fHigh > 0
        yBar = NaN;
        return;
    end

    yBar = fzero(scalarF, [yLow, yHigh]);

    res = scalarF(yBar);

    if ~isfinite(yBar) || abs(res) > args.tolerance
        yBar = NaN;
    end
end
```

---

9. Drop-in method 7 - state Gaussian at expiry

Under T_0-forward measure, at expiry T_0:

$$(X_{T_0}, Y_{T_0}) \sim \mathcal{N}\left(\mu^{T_0}, \Sigma\right).$$

```
function output = getStateGaussianAtExpiry(obj, currMaturity)
    % Computes the Gaussian law of (x(T0), y(T0)) under the T0-forward measure.

    arguments
        obj (1,1) GaussianTwoFactors;
        currMaturity (1,1) double;
    end

    T0 = currMaturity;

    muX = obj.muTForward(0, T0, ...
        'process', 'x_process', ...
        'tMaturity', T0);

    muY = obj.muTForward(0, T0, ...
        'process', 'y_process', ...
        'tMaturity', T0);

    varX = obj.varTForward(0, T0, 'process', 'x_process');
    varY = obj.varTForward(0, T0, 'process', 'y_process');
    covXY = obj.covTForward(0, T0);

    covMatrix = [varX, covXY; covXY, varY];

    output.muX = muX;
    output.muY = muY;
    output.varX = varX;
    output.varY = varY;
    output.covXY = covXY;
    output.covMatrix = covMatrix;
end
```

This requires `varTForward` and `covTForward` to be inside the class methods block.

---

10. Drop-in method 8 - conditional $Y|X = x$

For bivariate normal:

$$Y|X = x \sim \mathcal{N}(m_{Y|x}, s^2_{Y|x}),$$

with:

$$m_{Y|x} = \mu_Y + \frac{\mathrm{Cov}(X,Y)}{\mathrm{Var}(X)}(x - \mu_X),$$

$$s^2_{Y|x} = \mathrm{Var}(Y) - \frac{\mathrm{Cov}(X,Y)^2}{\mathrm{Var}(X)}.$$

```
function output = getConditionalYMoments(obj, x, gaussianState)
    % Computes conditional moments of Y given X = x.

    arguments
        obj (1,1) GaussianTwoFactors;
        x (1,1) double;
        gaussianState (1,1) struct;
    end

    varX = gaussianState.varX;
    varY = gaussianState.varY;
    covXY = gaussianState.covXY;

    if varX <= 0
        error("varX must be strictly positive for conditional Gaussian formul
a.");
    end

    condMeanY = gaussianState.muY + (covXY / varX) * (x - gaussianState.muX);
    condVarY = varY - (covXY^2 / varX);

    condVarY = max(condVarY, 0);

    output.meanY = condMeanY;
    output.varY = condVarY;
    output.stdY = sqrt(condVarY);
end
```

---

11. Main method - `priceSwaptionG2PP`

This is the immediate pricer.

It uses numerical integration over the normalized X-factor:

$$X = \mu_X + \sigma_X z, \qquad z \sim \mathcal{N}(0,1).$$

For payer:

$$\mathbb{E}[(1 - \sum_i \lambda_i e^{-B_i Y})^+ | X = x].$$

For receiver:

$$\mathbb{E}[(\sum_i \lambda_i e^{-B_i Y} - 1)^+ | X = x].$$

```
function price = priceSwaptionG2PP(obj, strike, currMaturity, ulSwapSteps, arg
s)
    % Prices a European swaption under G2++ using one-dimensional integration.
    %
    % Payer payoff at expiry T0:
    %
    %     [1 - sum_i c_i P(T0,Ti)]^+
    %
    % Receiver payoff:
    %
    %     [sum_i c_i P(T0,Ti) - 1]^+
    %
    % Price:
    %
    %     P(0,T0) * E^{T0}[ payoff ]

    arguments
        obj (1,1) GaussianTwoFactors;
        strike (1,1) double;
        currMaturity (1,1) double;
        ulSwapSteps (1,:) double;
        args.direction (1,:) char {mustBeMember(args.direction, {'PAY','REC'})}
= 'PAY';
        args.notional (1,1) double = 1e6;
        args.integrationLower (1,1) double = -8;
        args.integrationUpper (1,1) double = 8;
        args.absTol (1,1) double = 1e-10;
        args.relTol (1,1) double = 1e-8;
    end

    instr = obj.getSwaptionInstrument( ...
        strike, currMaturity, ulSwapSteps, ...
        'direction', args.direction, ...
        'notional', args.notional);

    basket = obj.getSwaptionBondBasket(instr.expiry, instr.paymentDates);

    gaussianState = obj.getStateGaussianAtExpiry(instr.expiry);

    if gaussianState.varX <= 0
        error("X variance at expiry must be strictly positive.");
    end

    stdX = sqrt(gaussianState.varX);

    coeffs = instr.coeffs(:);
    A0Ti = basket.A0Ti(:);
    Bx = basket.Bx0Ti(:);
```

```
        By = basket.By0Ti(:);

        integrand = @(z) localSwaptionIntegrand( ...
            z, ...
            gaussianState.muX, ...
            stdX, ...
            gaussianState, ...
            coeffs, A0Ti, Bx, By, ...
            instr.direction);

        expectation = integral( ...
            integrand, ...
            args.integrationLower, ...
            args.integrationUpper, ...
            'ArrayValued', true, ...
            'AbsTol', args.absTol, ...
            'RelTol', args.relTol);

        price = instr.notional * basket.P0T0 * expectation;
    end
```

This method calls a local helper. If your library dislikes local functions, make this a private method.

---

12. Helper - local integrand

Place this as a local function at the bottom of the file, or convert to a private method.

```
function val = localSwaptionIntegrand(z, muX, stdX, gaussianState, coeffs, A0T
i, Bx, By, direction)
    % Integrand over z ~ N(0,1).

    z = z(:).';
    val = zeros(size(z));

    for j = 1:numel(z)
        x = muX + stdX * z(j);

        lambda = coeffs .* A0Ti .* exp(-Bx .* x);

        yBar = localSolveYBarFromLambda(lambda, By);

        if ~isfinite(yBar)
            val(j) = NaN;
            continue;
        end

        varX = gaussianState.varX;
        varY = gaussianState.varY;
        covXY = gaussianState.covXY;

        meanY = gaussianState.muY + (covXY / varX) * (x - gaussianState.muX);
        varYCond = max(varY - covXY^2 / varX, 0);
        stdYCond = sqrt(varYCond);

        if stdYCond <= 0
            basketValue = sum(lambda .* exp(-By .* meanY));

            if strcmpi(direction, 'PAY')
                payoff = max(1 - basketValue, 0);
            else
                payoff = max(basketValue - 1, 0);
            end

            val(j) = payoff * normpdf(z(j));
            continue;
        end

        if strcmpi(direction, 'PAY')
            % payoff positive when Y > yBar
            probTerm = normcdf((meanY - yBar) / stdYCond);

            expTerm = exp(-By .* meanY + 0.5 .* (By.^2) .* varYCond) ...
                    .* normcdf((meanY - yBar - By .* varYCond) ./ stdYCond);

            condPayoff = probTerm - sum(lambda .* expTerm);
```

```
        else
            % receiver payoff positive when Y < yBar
            probTerm = normcdf((yBar - meanY) / stdYCond);

            expTerm = exp(-By .* meanY + 0.5 .* (By.^2) .* varYCond) ...
                    .* normcdf((yBar - meanY + By .* varYCond) ./ stdYCond);

            condPayoff = sum(lambda .* expTerm) - probTerm;
        end

        val(j) = max(condPayoff, 0) * normpdf(z(j));
    end
end
```

13. Helper - local y-bar solver from lambda

```
function yBar = localSolveYBarFromLambda(lambda, By)
    % Solves:
    %
    %     1 - sum_i lambda_i exp(-By_i y) = 0

    lambda = lambda(:);
    By = By(:);

    f = @(y) 1 - sum(lambda .* exp(-By .* y));

    yLow = -0.05;
    yHigh = 0.05;

    fLow = f(yLow);
    fHigh = f(yHigh);

    expansion = 0;

    while fLow * fHigh > 0 && expansion < 30
        width = yHigh - yLow;
        yLow = yLow - 2 * width;
        yHigh = yHigh + 2 * width;

        fLow = f(yLow);
        fHigh = f(yHigh);

        expansion = expansion + 1;
    end

    if fLow * fHigh > 0
        yBar = NaN;
        return;
    end

    yBar = fzero(f, [yLow, yHigh]);
end
```

---

14. Minimal insertion order

Do not insert `priceSwaptionG2PP` first. Insert in this order:

```
varI
```

```
fixed muTForward
```

```
varTForward and covTForward inside methods block
```

```
getSwaptionInstrument
```

```
getSwaptionBondBasket
```

```
getLambdaAndBy
```

```
getYBarValue
```

```
yBarSolver
```

```
getStateGaussianAtExpiry
```

```
priceSwaptionG2PP
```

Then test one ATM payer swaption.

---

15. First robustness tests

Test 1 - class method availability

```
methods(model)
```

Check that these appear:

```
varTForward
covTForward
getSwaptionInstrument
getSwaptionBondBasket
priceSwaptionG2PP
```

If not, they are still outside the class.

Test 2 - positive state covariance

```
G = model.getStateGaussianAtExpiry(5.0);

Sigma = G.covMatrix;

eigVals = eig(Sigma);

assert(all(eigVals > -1e-12));
```

This catches invalid ρ, bad vol buckets, or formula bugs.

Test 3 - positive finite swaption price

```
strike = 0.02;
expiry = 5.0;
paymentDates = 6.0:1.0:15.0;

price = model.priceSwaptionG2PP(strike, expiry, paymentDates, ...
    'direction', 'PAY', ...
    'notional', 1e6);

assert(isfinite(price));
assert(price >= 0);
```

---

16. Bottom line

Current class is close enough to attempt a first swaption price, but only after fixing the fatal plumbing:

$$\boxed{\text{class boundary}}$$

$$\boxed{\texttt{varI}/\texttt{vari}mismatch}$$

$$\boxed{\texttt{muTForward}optionalmaturityandsign}$$

$$\boxed{\texttt{getSwaptionInstrument}incomplete}$$

$$\boxed{\texttt{yBarSolver}undefinedfallback}$$

Then `priceSwaptionG2PP` can be introduced as a **prototype pricing method**. Calibration should come only after this method returns stable finite prices over a small swaption grid.

Response:

5/2/2026, 7:19:56 PM

> ***Evaluating and defining formulas with h1, h2, kappa***
>
> *I need to clarify the relationship between h1/h2/kappa in the formulas. I might propose using the priceSwaptionG2PP function and assess the formula. I also need to be careful with definitions—like possibly redefining kappa as ybar. I should include payer/receiver formulas and explain the conditional expectation, especially the negative sign flip for receiver payoff.*
>
> *Thought for 15s*

The method I proposed is the same conditional-Gaussian route, but I wrote it as an implementation derivation. If you already have `h1`, `h2`, and `kappa`, then we should use Brigo's explicit one-dimensional formula directly.

Yes. The method I proposed is **equivalent in structure**, but it was **not written in Brigo's explicit $h_1/h_2/\kappa$ notation**.

If you already coded:

```
h1Function
h2Function
kappa / yBar solver
```

then we should use those directly.

The correct next step is:

$$\boxed{\texttt{priceSwaptionG2PP} = \text{Brigo one-dimensional explicit swaption formula}}$$

not a generic conditional payoff implementation.

---

1. Object under discussion

For a payer swaption at expiry T_0, the payoff can be written as:

$$\left[1 - \sum_{i=1}^{n} c_i P(T_0, T_i)\right]^+ .$$

Under G2++:

$$P(T_0, T_i) = A_i \exp(-B_i^x x - B_i^y y).$$

So define:

$$\lambda_i(x) = c_i A_i \exp(-B_i^x x).$$

Then the root $\kappa(x)$, or $\bar{y}(x)$, solves:

$$1 = \sum_{i=1}^{n} \lambda_i(x) \exp(-B_i^y \kappa(x)).$$

This is exactly the scalar nonlinear equation you have been coding.

---

2. Brigo explicit conditional formula

Condition on $X = x$. Then:

$$Y|X = x \sim \mathcal{N}(m_{Y|x}, s_{Y|x}^2).$$

For a payer swaption, payoff is positive when:

$$Y > \kappa(x).$$

Define:

$$h_1(x) = \frac{m_{Y|x} - \kappa(x)}{s_{Y|x}},$$

and for each cash-flow date:

$$h_{2,i}(x) = \frac{m_{Y|x} - \kappa(x) - B_i^y s_{Y|x}^2}{s_{Y|x}}.$$

Then:

$$\mathbb{E}\left[\left(1 - \sum_i \lambda_i(x) e^{-B_i^y Y}\right)^+ \middle| X = x\right] = N(h_1(x)) - \sum_i \lambda_i(x) e^{-B_i^y m_{Y|x} + \frac{1}{2}(B_i^y)^2 s_{Y|x}^2} N(h_{2,i}(x)$$

Therefore:

$$\boxed{\text{PayerSwaption} = N_0 P(0, T_0) \int_{-\infty}^{+\infty} \left[N(h_1(x)) - \sum_i \lambda_i(x) e^{-B_i^y m_{Y|x} + \frac{1}{2}(B_i^y)^2 s_{Y|x}^2} N(h_{2,i}(x))\right]}$$

This is the Brigo-style formula.

The implementation I proposed earlier computes the same conditional expectation, but it did not explicitly call your `h1Function`, `h2Function`, and `kappa`.

---

3. Software responsibility

If you already have:

```
h1Function
h2Function
kappaFunction
```

then `priceSwaptionG2PP` should only orchestrate:

```
instrument
basket
Gaussian law at expiry
x integration
kappa(x)
h1(x), h2_i(x)
conditional payoff formula
discount by P(0,T0)
```

So the method stack should be:

```
priceSwaptionG2PP
    getSwaptionInstrument
    getSwaptionBondBasket
    getStateGaussianAtExpiry
    getLambdaAndBy
    kappaFunction / yBarSolver
    h1Function
    h2Function
```

---

4. Necessary correction to our previous method

Previously I wrote an internal formula like:

```
probTerm = normcdf((meanY - yBar) / stdYCond);

expTerm = exp(-By .* meanY + 0.5 .* (By.^2) .* varYCond) ...
        .* normcdf((meanY - yBar - By .* varYCond) ./ stdYCond);
```

This is exactly:

$$N(h_1(x)),$$

and:

$$e^{-B_i^y m_{Y|x} + \frac{1}{2}(B_i^y)^2 s_{Y|x}^2} N(h_{2,i}(x)).$$

So yes, the formula is Brigo-compatible. But the implementation should be rewritten to expose this structure explicitly.

---

5. Proposed `priceSwaptionG2PP` using h_1, h_2, κ

Use this as the cleaner method.

```
function price = priceSwaptionG2PP(obj, strike, currMaturity, ulSwapSteps, arg
s)
    % Prices a European swaption under G2++ using the Brigo-style
    % one-dimensional explicit formula.
    %
    % Payer payoff:
    %
    %     [1 - sum_i c_i P(T0,Ti)]^+
    %
    % Receiver payoff:
    %
    %     [sum_i c_i P(T0,Ti) - 1]^+
    %
    % The method integrates over the marginal Gaussian law of x(T0)
    % under the T0-forward measure.

    arguments
        obj (1,1) GaussianTwoFactors;
        strike (1,1) double;
        currMaturity (1,1) double;
        ulSwapSteps (1,:) double;
        args.direction (1,:) char {mustBeMember(args.direction, {'PAY','REC'})}
= 'PAY';
        args.notional (1,1) double = 1e6;
        args.integrationLower (1,1) double = -8;
        args.integrationUpper (1,1) double = 8;
        args.absTol (1,1) double = 1e-10;
        args.relTol (1,1) double = 1e-8;
    end

    % 1. Build standardized instrument and affine bond basket
    instr = obj.getSwaptionInstrument( ...
        strike, currMaturity, ulSwapSteps, ...
        'direction', args.direction, ...
        'notional', args.notional);

    basket = obj.getSwaptionBondBasket(instr.expiry, instr.paymentDates);

    % 2. Gaussian law of (x(T0), y(T0)) under T0-forward measure
    gaussianState = obj.getStateGaussianAtExpiry(instr.expiry);

    muX = gaussianState.muX;
    varX = gaussianState.varX;
    stdX = sqrt(varX);

    if varX <= 0
        error("X variance at expiry must be strictly positive.");
    end
```

```
    % 3. Extract fixed basket objects
    coeffs = instr.coeffs(:);
    A0Ti = basket.A0Ti(:);
    Bx = basket.Bx0Ti(:);
    By = basket.By0Ti(:);

    % 4. Integrate over standardized z, x = muX + stdX*z
    integrand = @(z) obj.swaptionBrigoIntegrand( ...
        z, ...
        muX, ...
        stdX, ...
        gaussianState, ...
        coeffs, ...
        A0Ti, ...
        Bx, ...
        By, ...
        instr.direction);

    expectation = integral( ...
        integrand, ...
        args.integrationLower, ...
        args.integrationUpper, ...
        'ArrayValued', true, ...
        'AbsTol', args.absTol, ...
        'RelTol', args.relTol);

    price = instr.notional * basket.P0T0 * expectation;
end
```

---

6. Add method `swaptionBrigoIntegrand`

This is where your h_1, h_2, κ should enter.

```
function output = swaptionBrigoIntegrand(obj, z, muX, stdX, gaussianState, coef
fs, A0Ti, Bx, By, direction)
    % Brigo-style G2++ swaption integrand over z ~ N(0,1).
    %
    % x = muX + stdX*z.

    arguments
        obj (1,1) GaussianTwoFactors;
        z double;
        muX (1,1) double;
        stdX (1,1) double;
        gaussianState (1,1) struct;
        coeffs (:,1) double;
        A0Ti (:,1) double;
        Bx (:,1) double;
        By (:,1) double;
        direction (1,:) char {mustBeMember(direction, {'PAY','REC'})};
    end

    z = z(:).';
    output = zeros(size(z));

    for j = 1:numel(z)

        xVal = muX + stdX * z(j);

        % lambda_i(x)
        lambda = coeffs .* A0Ti .* exp(-Bx .* xVal);

        % kappa(x), i.e. yBar(x)
        kappaVal = obj.kappaFunction(lambda, By);

        if ~isfinite(kappaVal)
            output(j) = NaN;
            continue;
        end

        % Conditional Gaussian law of Y | X = x
        cond = obj.getConditionalYMoments(xVal, gaussianState);

        muYCond = cond.meanY;
        varYCond = cond.varY;
        stdYCond = cond.stdY;

        if stdYCond <= 0
            basketValue = sum(lambda .* exp(-By .* muYCond));

            if strcmpi(direction, 'PAY')
```

```
                condPayoff = max(1 - basketValue, 0);
            else
                condPayoff = max(basketValue - 1, 0);
            end

            output(j) = condPayoff * normpdf(z(j));
            continue;
        end

        % h1 and h2 in Brigo notation
        h1 = obj.h1Function(kappaVal, muYCond, stdYCond);

        h2 = obj.h2Function(kappaVal, muYCond, stdYCond, By);

        bondExponential = exp(-By .* muYCond + 0.5 .* (By.^2) .* varYCond);

        if strcmpi(direction, 'PAY')
            condPayoff = normcdf(h1) - sum(lambda .* bondExponential .* normcdf
(h2));
        else
            % Receiver formula: lower tail instead of upper tail
            condPayoff = sum(lambda .* bondExponential .* normcdf(-h2)) - normc
df(-h1);
        end

        output(j) = max(condPayoff, 0) * normpdf(z(j));
    end
end
```

This assumes the following convention:

$$h_1 = \frac{\mu_{Y|x} - \kappa}{\sigma_{Y|x}},$$

$$h_{2,i} = \frac{\mu_{Y|x} - \kappa - B_i^y \sigma_{Y|x}^2}{\sigma_{Y|x}}.$$

If your existing `h1Function` and `h2Function` use the opposite sign, then the payer/receiver tails must be flipped.

7. Required signatures for your existing helpers

Your existing functions should ideally match these signatures.

`kappaFunction`

```
function kappa = kappaFunction(obj, lambda, By)
```

Solves:

$$1 - \sum_i \lambda_i e^{-B_i^y \kappa} = 0.$$

`h1Function`

```
function output = h1Function(obj, kappa, muYCond, stdYCond)
```

Returns:

$$h_1 = \frac{\mu_{Y|x} - \kappa}{\sigma_{Y|x}}.$$

`h2Function`

```
function output = h2Function(obj, kappa, muYCond, stdYCond, By)
```

Returns vector:

$$h_{2,i} = \frac{\mu_{Y|x} - \kappa - B_i^y \sigma_{Y|x}^2}{\sigma_{Y|x}}.$$

---

8. What to change from previous proposed code

Replace the earlier generic conditional expectation block with this explicit structure:

```
kappaVal = obj.kappaFunction(lambda, By);

h1 = obj.h1Function(kappaVal, muYCond, stdYCond);

h2 = obj.h2Function(kappaVal, muYCond, stdYCond, By);

bondExponential = exp(-By .* muYCond + 0.5 .* (By.^2) .* varYCond);

condPayoff = normcdf(h1) - sum(lambda .* bondExponential .* normcdf(h2));
```

This is much cleaner and closer to Brigo.

---

9. Main caution

The whole formula depends on the sign convention of κ, h_1, and h_2.

For payer, with the convention:

$$\text{payoff positive when } Y > \kappa(x),$$

we need:

$$N(h_1) = \mathbb{P}(Y > \kappa(x) \mid X = x).$$

Since:

$$\mathbb{P}(Y > \kappa) = N\left(\frac{\mu - \kappa}{\sigma}\right),$$

the correct payer convention is:

$$h_1 = \frac{\mu - \kappa}{\sigma}.$$

If your code defines:

$$h_1 = \frac{\kappa - \mu}{\sigma},$$

then use:

```
normcdf(-h1)
```

instead.

Same for h_2.

---

10. Bottom line

Yes: we should use Brigo's explicit formula.

The previous method was a valid conditional-Gaussian implementation, but for your codebase the better next step is:

$$\boxed{\texttt{priceSwaptionG2PP} \to \texttt{swaptionBrigoIntegrand} \to \texttt{kappaFunction} + \texttt{h1Function}}$$

That aligns the implementation with the theory and makes the method easier to audit.

Response:

5/2/2026, 7:25:20 PM

Yes. With the theorem you showed, use **Brigo's notation exactly**.

Key convention:

$$\omega = \begin{cases} 1, & \text{payer}, \\ -1, & \text{receiver}. \end{cases}$$

and

$$\text{Price} = NP(0,T) \int f_X(x)\, \omega \left[\Phi(-\omega h_1(x)) - \sum_i \lambda_i(x) e^{\kappa_i(x)} \Phi(-\omega h_{2,i}(x)) \right] dx.$$

Below is the code starting from the `yBarSolver` block, assuming:

```
getSwaptionInstrument
getSwaptionBondBasket
getStateGaussianAtExpiry
varI
muTForward
varTForward
covTForward
```

are already fixed and inside the class.

1. Scalar equation for $\bar{y}(x)$

```
function value = getYBarValue(obj, y, lambda, By)
    % Scalar equation for yBar:
    %
    %     f(y) = 1 - sum_i lambda_i(x) * exp(-By_i * y)
    %
    % where:
    %
    %     lambda_i(x) = c_i * A(T,T_i) * exp(-Bx_i * x)

    arguments
        obj (1,1) GaussianTwoFactors;
        y (1,1) double;
        lambda (:,1) double;
        By (:,1) double;
    end

    value = 1 - sum(lambda .* exp(-By .* y));
end
```

2. Robust $\bar{y}(x)$ solver

This version receives already-computed $\lambda_i(x)$ and B_i^y. This is cleaner for Brigo's formula.

```
function yBar = yBarSolverFromLambda(obj, lambda, By, args)
    % Solves for yBar(x):
    %
    %     1 = sum_i lambda_i(x) * exp(-By_i * yBar)
    %
    % This is the Brigo \bar{y}(x) equation.

    arguments
        obj (1,1) GaussianTwoFactors;
        lambda (:,1) double;
        By (:,1) double;
        args.ySeed (1,1) double = 0;
        args.maxExpansion (1,1) double = 30;
        args.tolerance (1,1) double = 1e-10;
    end

    lambda = lambda(:);
    By = By(:);

    scalarF = @(y) obj.getYBarValue(y, lambda, By);

    yLow = args.ySeed - 0.01;
    yHigh = args.ySeed + 0.01;

    fLow = scalarF(yLow);
    fHigh = scalarF(yHigh);

    expansion = 0;

    while fLow * fHigh > 0 && expansion < args.maxExpansion
        width = yHigh - yLow;

        yLow = yLow - 2 * width;
        yHigh = yHigh + 2 * width;

        fLow = scalarF(yLow);
        fHigh = scalarF(yHigh);

        expansion = expansion + 1;
    end

    if fLow * fHigh > 0
        yBar = NaN;
        return;
    end

    yBar = fzero(scalarF, [yLow, yHigh]);
```

```
    residual = scalarF(yBar);

    if ~isfinite(yBar) || abs(residual) > args.tolerance
        yBar = NaN;
    end
end
```

Optional wrapper preserving your old interface:

```
function yBar = yBarSolver(obj, strike, currMaturity, ulSwapSteps, x_state, arg
s)
    % Wrapper around yBarSolverFromLambda using instrument inputs.

    arguments
        obj (1,1) GaussianTwoFactors;
        strike (1,1) double;
        currMaturity (1,1) double;
        ulSwapSteps (1,:) double;
        x_state (1,1) double = 0;
        args.direction (1,:) char {mustBeMember(args.direction, {'PAY','REC'})}
= 'PAY';
        args.notional (1,1) double = 1e6;
        args.ySeed (1,1) double = 0;
    end

    [lambda, By] = obj.getLambdaAndBy( ...
        strike, currMaturity, ulSwapSteps, x_state, ...
        'direction', args.direction, ...
        'notional', args.notional);

    yBar = obj.yBarSolverFromLambda(lambda, By, 'ySeed', args.ySeed);
end
```

---

3. Brigo $h_1(x)$

Use the theorem convention:

$$h_1(x) = \frac{\bar{y}(x) - \mu_{Y|x}}{\sigma_{Y|x}}.$$

```
function h1 = h1Function(obj, yBar, muYCond, sigmaYCond)
    % Brigo h1:
    %
    %     h1(x) = (yBar(x) - E[Y|X=x]) / Std[Y|X=x]

    arguments
        obj (1,1) GaussianTwoFactors;
        yBar (1,1) double;
        muYCond (1,1) double;
        sigmaYCond (1,1) double {mustBeNonnegative};
    end

    if sigmaYCond <= 0
        error("sigmaYCond must be strictly positive.");
    end

    h1 = (yBar - muYCond) ./ sigmaYCond;
end
```

---

4. Brigo $h_2(x)$

The theorem uses:

$$h_{2,i}(x) = h_1(x) + B(b, T, T_i)\sigma_{Y|x}.$$

```
function h2 = h2Function(obj, h1, By, sigmaYCond)
    % Brigo h2:
    %
    %     h2_i(x) = h1(x) + By_i * Std[Y|X=x]

    arguments
        obj (1,1) GaussianTwoFactors;
        h1 (1,1) double;
        By (:,1) double;
        sigmaYCond (1,1) double {mustBeNonnegative};
    end

    h2 = h1 + By(:) .* sigmaYCond;
end
```

---

5. Brigo $\kappa_i(x)$

The theorem uses:

$$\kappa_i(x) = -B_i^y \left[\mu_{Y|x} - \frac{1}{2} B_i^y \sigma_{Y|x}^2 \right].$$

This is equivalent to:

$$\kappa_i(x) = -B_i^y \mu_{Y|x} + \frac{1}{2} (B_i^y)^2 \sigma_{Y|x}^2.$$

```
function kappa = kappaFunction(obj, By, muYCond, varYCond)
    % Brigo kappa_i(x):
    %
    %     kappa_i(x)
    %       = -By_i * muYCond + 0.5 * By_i^2 * varYCond
    %
    % where:
    %
    %     muYCond = E[Y | X = x]
    %     varYCond = Var[Y | X = x]

    arguments
        obj (1,1) GaussianTwoFactors;
        By (:,1) double;
        muYCond (1,1) double;
        varYCond (1,1) double {mustBeNonnegative};
    end

    By = By(:);

    kappa = -By .* muYCond + 0.5 .* (By.^2) .* varYCond;
end
```

---

6. Brigo integrand

This is the core implementation of the theorem.

We integrate over:

$$z \sim \mathcal{N}(0,1), \qquad x = \mu_x + \sigma_x z.$$

So the density is `normpdf(z)`.

```
function output = brigoSwaptionIntegrand(obj, z, omega, coeffs, A0Ti, Bx, By, g
aussianState)
    % Brigo G2++ swaption integrand using standardized normal variable z.
    %
    % x = muX + sigmaX * z
    %
    % Integrand:
    %
    % omega * [
    %     Phi(-omega*h1)
    %     - sum_i lambda_i(x) * exp(kappa_i(x)) * Phi(-omega*h2_i)
    % ] * phi(z)

    arguments
        obj (1,1) GaussianTwoFactors;
        z double;
        omega (1,1) double {mustBeMember(omega, [-1, 1])};
        coeffs (:,1) double;
        A0Ti (:,1) double;
        Bx (:,1) double;
        By (:,1) double;
        gaussianState (1,1) struct;
    end

    z = z(:).';
    output = zeros(size(z));

    muX = gaussianState.muX;
    muY = gaussianState.muY;
    varX = gaussianState.varX;
    varY = gaussianState.varY;
    covXY = gaussianState.covXY;

    sigmaX = sqrt(varX);

    if sigmaX <= 0
        error("sigmaX must be strictly positive.");
    end

    coeffs = coeffs(:);
    A0Ti = A0Ti(:);
    Bx = Bx(:);
    By = By(:);

    for j = 1:numel(z)

        xVal = muX + sigmaX * z(j);
```

```
        % lambda_i(x)
        lambda = coeffs .* A0Ti .* exp(-Bx .* xVal);

        % yBar(x)
        yBar = obj.yBarSolverFromLambda(lambda, By);

        if ~isfinite(yBar)
            output(j) = NaN;
            continue;
        end

        % Conditional Gaussian Y | X = x
        muYCond = muY + (covXY / varX) * (xVal - muX);
        varYCond = varY - (covXY^2 / varX);
        varYCond = max(varYCond, 0);
        sigmaYCond = sqrt(varYCond);

        if sigmaYCond <= 0
            basketVal = sum(lambda .* exp(-By .* muYCond));

            if omega == 1
                condPayoff = max(1 - basketVal, 0);
            else
                condPayoff = max(basketVal - 1, 0);
            end

            output(j) = condPayoff * normpdf(z(j));
            continue;
        end

        % Brigo objects
        h1 = obj.h1Function(yBar, muYCond, sigmaYCond);
        h2 = obj.h2Function(h1, By, sigmaYCond);
        kappa = obj.kappaFunction(By, muYCond, varYCond);

        bracket = normcdf(-omega .* h1) ...
                - sum(lambda .* exp(kappa) .* normcdf(-omega .* h2));

        output(j) = omega .* bracket .* normpdf(z(j));
    end
end
```

7. Final `priceSwaptionG2PP`

```
function price = priceSwaptionG2PP(obj, strike, currMaturity, ulSwapSteps, arg
s)
    % Prices a European swaption under G2++ using Brigo's explicit
    % one-dimensional integral.
    %
    % Theorem convention:
    %
    %     omega =  1 for payer
    %     omega = -1 for receiver

    arguments
        obj (1,1) GaussianTwoFactors;
        strike (1,1) double;
        currMaturity (1,1) double;
        ulSwapSteps (1,:) double;
        args.direction (1,:) char {mustBeMember(args.direction, {'PAY','REC'})}
= 'PAY';
        args.notional (1,1) double = 1e6;
        args.integrationLower (1,1) double = -8;
        args.integrationUpper (1,1) double = 8;
        args.absTol (1,1) double = 1e-10;
        args.relTol (1,1) double = 1e-8;
    end

    if strcmpi(args.direction, 'PAY')
        omega = 1;
    else
        omega = -1;
    end

    % Instrument coefficients c_i
    instr = obj.getSwaptionInstrument( ...
        strike, currMaturity, ulSwapSteps, ...
        'direction', args.direction, ...
        'notional', args.notional);

    % Affine bond basket A(T,T_i), Bx(T,T_i), By(T,T_i)
    basket = obj.getSwaptionBondBasket(instr.expiry, instr.paymentDates);

    % Gaussian law of (x_T, y_T) under T-forward measure
    gaussianState = obj.getStateGaussianAtExpiry(instr.expiry);

    coeffs = instr.coeffs(:);
```

```
        A0Ti = basket.A0Ti(:);
        Bx = basket.Bx0Ti(:);
        By = basket.By0Ti(:);

        integrand = @(z) obj.brigoSwaptionIntegrand( ...
            z, omega, coeffs, A0Ti, Bx, By, gaussianState);

        expectation = integral( ...
            integrand, ...
            args.integrationLower, ...
            args.integrationUpper, ...
            'ArrayValued', true, ...
            'AbsTol', args.absTol, ...
            'RelTol', args.relTol);

        price = instr.notional .* basket.P0T0 .* expectation;

        if ~isfinite(price)
            price = NaN;
        end
    end
```

---

8. Critical convention check

This code uses the theorem's convention:

$$h_1(x) = \frac{\bar{y}(x) - \mu_{Y|x}}{\sigma_{Y|x}},$$

so the payer tail is:

$$\Phi(-h_1).$$

This is the opposite sign from the earlier conditional-expectation version I gave. With your screenshot, this is the correct one.

The object $\kappa_i(x)$ here is **not** $\bar{y}(x)$. It is the exponential adjustment:

$$\kappa_i(x) = -B_i^y \mu_{Y|x} + \frac{1}{2}(B_i^y)^2 \sigma_{Y|x}^2.$$

That distinction matters.

Response:

5/2/2026, 7:32:36 PM

> ***Proving h1 and h2 equivalence with conditional moments***
>
> *To prove h1 and h2 equivalence, we need to show conditional moments align. With a bivariate normal distribution and conditional mean and variance, I'll connect the equations. h1 is derived using variance, while h2 comes from truncated lognormal Gaussian expectation. The condition matches when adjusting for B and variance terms.*
>
> *Thought for 7s*

Let

$$(X, Y) \sim \mathcal{N}\left(\begin{pmatrix}\mu_x \\ \mu_y\end{pmatrix}, \begin{pmatrix}\sigma_x^2 & \rho_{xy}\sigma_x\sigma_y \\ \rho_{xy}\sigma_x\sigma_y & \sigma_y^2\end{pmatrix}\right).$$

In the G2++ swaption formula, $X = x_T$, $Y = y_T$, and the integration is done over x. Therefore the key object is the conditional law:

$$Y \mid X = x.$$

---

1. Conditional moments

For a bivariate Gaussian,

$$Y \mid X = x \sim \mathcal{N}(m_{Y|x}, s_{Y|x}^2),$$

with

$$m_{Y|x} = \mu_y + \frac{\mathrm{Cov}(X, Y)}{\mathrm{Var}(X)}(x - \mu_x).$$

Since

$$\mathrm{Cov}(X, Y) = \rho_{xy}\sigma_x\sigma_y, \qquad \mathrm{Var}(X) = \sigma_x^2,$$

we get

$$m_{Y|x} = \mu_y + \rho_{xy}\frac{\sigma_y}{\sigma_x}(x - \mu_x).$$

The conditional variance is

$$s_{Y|x}^2 = \sigma_y^2 - \frac{\mathrm{Cov}(X,Y)^2}{\mathrm{Var}(X)}.$$

Thus

$$s_{Y|x}^2 = \sigma_y^2 - \frac{\rho_{xy}^2 \sigma_x^2 \sigma_y^2}{\sigma_x^2} = \sigma_y^2 (1 - \rho_{xy}^2).$$

Therefore

$$s_{Y|x} = \sigma_y \sqrt{1 - \rho_{xy}^2}.$$

---

2. Proof of $h_1(x)$

By definition, Brigo's h_1 is the standardized distance between the exercise boundary $\bar{y} = \bar{y}(x)$ and the conditional mean of $Y \mid X = x$:

$$h_1(x) = \frac{\bar{y}(x) - m_{Y|x}}{s_{Y|x}}.$$

Substitute the conditional moments:

$$h_1(x) = \frac{\bar{y} - \left[\mu_y + \rho_{xy} \frac{\sigma_y}{\sigma_x}(x - \mu_x)\right]}{\sigma_y \sqrt{1 - \rho_{xy}^2}}.$$

Separate the terms:

$$h_1(x) = \frac{\bar{y} - \mu_y}{\sigma_y \sqrt{1 - \rho_{xy}^2}} - \frac{\rho_{xy} \frac{\sigma_y}{\sigma_x}(x - \mu_x)}{\sigma_y \sqrt{1 - \rho_{xy}^2}}.$$

Cancel σ_y:

$$\boxed{h_1(x) = \frac{\bar{y} - \mu_y}{\sigma_y \sqrt{1 - \rho_{xy}^2}} - \frac{\rho_{xy}(x - \mu_x)}{\sigma_x \sqrt{1 - \rho_{xy}^2}}}$$

which is exactly the formula in the text.

3. Proof of $h_2(x)$

Let

$$B_i := B(b, T, t_i).$$

In the swaption formula, terms of the form

$$\mathbb{E}\left[e^{-B_i Y}\mathbf{1}_{\{Y>\bar{y}\}} \mid X = x\right]$$

appear for the payer case.

Since

$$Y \mid X = x \sim \mathcal{N}(m_{Y|x}, s_{Y|x}^2),$$

we use the standard Gaussian exponential-tilting identity:

$$\mathbb{E}\left[e^{-B_i Y}\mathbf{1}_{\{Y>\bar{y}\}} \mid X = x\right] = e^{-B_i m_{Y|x} + \frac{1}{2}B_i^2 s_{Y|x}^2}\Phi\left(\frac{m_{Y|x} - \bar{y} - B_i s_{Y|x}^2}{s_{Y|x}}\right).$$

Now define

$$h_{2,i}(x)$$

by

$$\Phi(-h_{2,i}(x)) = \Phi\left(\frac{m_{Y|x} - \bar{y} - B_i s_{Y|x}^2}{s_{Y|x}}\right).$$

Therefore

$$-h_{2,i}(x) = \frac{m_{Y|x} - \bar{y} - B_i s_{Y|x}^2}{s_{Y|x}},$$

so

$$h_{2,i}(x) = \frac{\bar{y} - m_{Y|x} + B_i s_{Y|x}^2}{s_{Y|x}}.$$

But

$$h_1(x) = \frac{\bar{y} - m_{Y|x}}{s_{Y|x}},$$

hence

$$h_{2,i}(x) = h_1(x) + B_i s_{Y|x}.$$

Since

$$s_{Y|x} = \sigma_y \sqrt{1 - \rho_{xy}^2},$$

we obtain

$$\boxed{h_{2,i}(x) = h_1(x) + B(b, T, t_i)\sigma_y \sqrt{1 - \rho_{xy}^2}}$$

which is the stated formula.

---

4. MATLAB convention

So the clean implementation is:

```
function h1 = h1Function(obj, yBar, muY, sigmaY, rhoXY, xVal, muX, sigmaX)
    denom = sigmaY * sqrt(1 - rhoXY^2);

    h1 = ((yBar - muY) ./ denom) ...
       - ((rhoXY .* (xVal - muX)) ./ (sigmaX .* sqrt(1 - rhoXY^2)));
end
```

and

```
function h2 = h2Function(obj, h1, By, sigmaY, rhoXY)
    h2 = h1 + By(:) .* sigmaY .* sqrt(1 - rhoXY^2);
end
```

Equivalent conditional-moment version:

```
function h1 = h1FromConditionalMoments(obj, yBar, muYCond, sigmaYCond)
    h1 = (yBar - muYCond) ./ sigmaYCond;
end

function h2 = h2FromConditionalMoments(obj, h1, By, sigmaYCond)
    h2 = h1 + By(:) .* sigmaYCond;
end
```

The two forms are mathematically identical because

$$\mu_{Y|x} = \mu_y + \rho_{xy}\frac{\sigma_y}{\sigma_x}(x - \mu_x), \qquad \sigma_{Y|x} = \sigma_y\sqrt{1 - \rho_{xy}^2}.$$